



Syntactic multilingual probing of pre-trained language models of code[☆]

José Antonio Hernández López^a,^{*}, Martin Weyssow^b, Jesús Sánchez Cuadrado^a,
Houari Sahraoui^c

^a DIS, University of Murcia, Spain

^b Singapore Management University, Singapore

^c DIRO, University of Montreal, Canada

ARTICLE INFO

Dataset link: <https://github.com/Antolin1/multilingual-ast-probe>

Keywords:

Pre-trained language models

Abstract syntax tree

Probing

Programming languages

Cross-lingual

ABSTRACT

Pre-trained language models (PLMs) have demonstrated remarkable abilities in coding tasks, establishing themselves as a state-of-the-art technique in machine learning for code. However, due to their deep neural network-based structure, PLMs function as black-box systems, making it crucial to understand the types of information they actually learn. Recent studies indicate that PLMs possess cross-lingual capabilities, allowing them to generalize to unseen programming languages and outperform monolingual models when trained in a multilingual setting. Nonetheless, the reasons behind these cross-lingual abilities remain largely uncharted and remain open questions.

In this paper, we explore this phenomenon through a syntactic perspective. Specifically, we build on our prior work, the AST-Probe, a probing methodology that evaluates whether a PLM encodes the complete grammatical structure of a programming language. This probe identifies a *syntactic subspace* within the PLM's vector representations, which is then used to reconstruct ASTs. We extend this approach in two ways. First, we conducted experiments on eight programming languages and eight PLMs and found that: (1) this syntactic structure can be extracted in all cases, (2) CodeBERT and GraphCodeBERT excel at encoding ASTs, and (3) syntactic knowledge resides in the middle layers of all PLMs, with a distribution that is independent of the programming language. Secondly, we mathematically adapt the AST-Probe to a multilingual setting and apply it to CodeBERT. Our findings provide evidence that CodeBERT learns cross-lingual representations of programming languages syntax.

1. Introduction

Pre-trained language models (PLMs) have become increasingly important in the deep learning for code research field (Feng et al., 2020; Wang et al., 2021; Ahmad et al., 2021; Chen et al., 2021). They show prominent generalization capabilities on a broad range of downstream tasks and continue to gain in performance according to recent studies (Xu et al., 2022; Rabin et al., 2023). With PLMs playing such a central role in deep learning for code, understanding what information these models learn about source code naturally emerged as a fundamental question. In this context, previous works showed that PLMs of code learn syntactic and semantic properties of source code (López et al., 2022; Troshin and Chirkova, 2022; Karmakar and Robbes, 2021; Wan et al., 2022).

Emerging research highlights the transferability between programming languages, a capability effectively leveraged by PLMs. Notably, multilingual PLMs show strong generalization to previously unseen

programming languages (Feng et al., 2020; Lu et al., 2021; Ahmed and Devanbu, 2022). Moreover, multilingual fine-tuning techniques have been shown to outperform monolingual fine-tuning across various coding tasks (Ahmed and Devanbu, 2022). These findings hint at the possibility that PLMs may acquire a unified internal representation for multiple programming languages. Verifying this hypothesis is scientifically valuable because it exposes the inductive biases of Transformer encoder pre-training and shows whether language-agnostic abstractions, akin to a “universal grammar” for code, emerge automatically during pre-training. This concept draws inspiration from Chomsky's universal grammar theory in linguistics (Chomsky, 2014), which posits that certain grammatical principles are common across all human languages.

Our prior work introduced the *AST-Probe* (López et al., 2022), which uncovered low-dimensional *syntactic subspaces* in which rich information related to the abstract syntax tree (AST) of programming

[☆] Editor: Dr. Lingxiao Jiang.

^{*} Corresponding author.

E-mail address: joseantonio.hernandez6@um.es (J.A. Hernández López).

languages is encoded in PLMs. In turn, our work demonstrates that by exclusively leveraging information from this syntactic subspace, the AST of a code snippet can be reconstructed with high accuracy in models like CodeBERT. This previous work provides a significantly deeper understanding of the information encoded in the hidden representation spaces of PLMs. However, like other recent probing techniques (Ma et al., 2024; Wan et al., 2022), the AST-Probe is monolingual and assumes that each language occupies its own region of representation space. Thus we remain ignorant as to whether a *shared* cross-lingual subspace underpins the syntax of different languages in models such as CodeBERT.

In this paper, we bridge this gap by extending the AST-Probe into a *multilingual* probe that operates on several languages simultaneously and quantifies the degree to which their syntactic subspaces overlap. We replicate the experiments conducted in our earlier paper while considering a broader range of PLMs and programming languages. On top of these results, we extend the AST-Probe to a multilingual setting. By employing this multilingual probe and utilizing CodeBERT (Feng et al., 2020) as our representative PLM, we investigate whether CodeBERT acquires a cross-lingual representation of the syntactic structure of code in the pre-training phase. More concretely, we extend the AST-Probe to be applied in two scenarios:

- **Direct transfer $i \rightarrow j$.** We recover ASTs of a language j using the syntactic subspace of the language i . The rationale behind this approach is to determine whether the syntactic subspace of language i encodes valuable information about the syntax of language j . If successful to a significant extent, this would provide evidence for the existence of a cross-lingual representation of syntax.
- **Identifying a shared syntactic subspace.** In this scenario, our goal is to identify a common syntactic subspace by training the AST-Probe using a combination of code snippets from multiple languages. If that geometric structure exists and allows recovering ASTs of all input languages, it would provide additional evidence supporting the cross-lingual nature of the syntactic information embedded within CodeBERT.

By conducting these two analyses, we aim to deepen our understanding of the presence and characteristics of cross-lingual representations within CodeBERT. This research contributes to our knowledge of how syntactic knowledge transcends individual programming languages and is encoded within PLMs.

Altogether, this paper makes two significant contributions:

1. **Multilingual AST-Probe.** We extend AST-Probe to a multilingual context and apply it to CodeBERT, demonstrating its ability to learn cross-lingual representations of code syntax. In a direct transfer scenario, we show that the syntactic structures (ASTs) of multiple programming languages can be recovered using the syntactic subspace of other languages. Moreover, ASTs can be extracted from a shared syntactic subspace that spans all the languages considered.
2. **Large-scale empirical study.** We evaluate the AST-Probe across eight programming languages: Python, Java, Ruby, Go, JavaScript, PHP, C, and C#, and eight PLMs: BERT (Devlin et al., 2018), DistilBERT (Sanh et al., 2019), RoBERTa (Liu et al., 2019), DistilRoBERTa (Wolf et al., 2019), CodeT5 (Wang et al., 2021), CodeBERTa (Wolf et al., 2019), CodeBERT (Feng et al., 2020), and GraphCodeBERT (Guo et al., 2020). Our analysis reveals that the hidden representations of these PLMs encode significantly more syntactic information compared to the baseline.

Organization. Section 2 introduces preliminaries on PLMs and related work on probing in natural language processing (NLP) and deep

learning for code. In Section 3, we describe the AST-Probe method. Section 4 introduces our novel multilingual probe. In Sections 5 and 6, we go through our experimental setup and discuss the results of our experiments. Section 7 presents a case study that shows the application of the probing methodologies introduced in this paper. We address some threats to the validity of our experiments in Section 8. Finally, we close the paper with a discussion in Section 9 and a conclusion in Section 10.

2. Background and related work

In this section, we introduce pretrained language models (PLMs) and outline the notations used throughout the paper. We also review related work on probing, with a focus on probing techniques in deep learning for code.

2.1. Pre-trained language models and notations

PLMs are built upon the Transformer architecture (Vaswani et al., 2017). Following a prior study (Min et al., 2021), PLMs can be roughly categorized into three categories depending on their pre-training objective. *Decoder* models such as GPT-3 (Brown et al., 2020), CodeGPT (Lu et al., 2021), or StarCoder (Li et al., 2023) whose training objective is to predict the next token in a sequence given the previous ones in an autoregressive manner. *Encoder* models include BERT (Devlin et al., 2018), RoBERTa (Liu et al., 2019) or CodeBERT (Feng et al., 2020) and leverage learning objectives such as masked language modeling to learn good representations of the input sequences. Finally, *encoder-decoder* models are generally pre-trained on conditional generation or sentence reconstruction learning objectives and include models like T5 (Raffel et al., 2019) or CodeT5 (Wang et al., 2021).

This paper focuses exclusively on encoder-based PLMs, which are particularly effective for classification tasks and those that require feature extraction, such as code search (Husain et al., 2019) and code clone detection (Guo et al., 2020). Furthermore, these models usually learn richer code representations thanks to their bidirectional pre-training approach (Devlin et al., 2018).

Following the notations of López et al. (2022), an encoder-based PLM¹ maps a sequence of input tokens into a set of contextualized vectors. Let us consider $w_0, \dots, w_n$ a sequence of $n + 1$ tokens and an encoder consisting of $\mathcal{L}$ transformer layers. The hidden vectors of the layer $1 \leq l \leq \mathcal{L}$ associated with the input sequence are computed in the following way:

$$H^\ell = \text{Transformer}_\ell(H^{\ell-1}), \quad (1)$$

where $H^\ell = [h_0^\ell, \dots, h_n^\ell]$, Transformer_ℓ is a transformer layer based on multi-head attention (Vaswani et al., 2017). H^0 are the initial embeddings computed as a sum of the positional encodings and the initial token embeddings. In this paper, we aim to study the syntactic information stored in the hidden vectors $h_i^\ell \in H^\ell$ for $\ell = 1, \dots, \mathcal{L}$.

2.2. Probing

Probing is a technique that utilizes probes, also known as diagnosis classifiers (Belinkov, 2016), to evaluate the encoding of specific properties of interest in the hidden representation spaces of a neural network. A probe classifier is a machine learning model that takes vector representations as input and predicts the property of interest. Probing is a well-established method in NLP to analyze PLMs and the properties of the natural languages they encode. Recently, probes have also been applied in the field of deep learning for code to investigate the understanding of source code by PLMs.

¹ For simplicity, we will use *PLM* to refer specifically to encoder-based PLMs throughout the remainder of this manuscript.

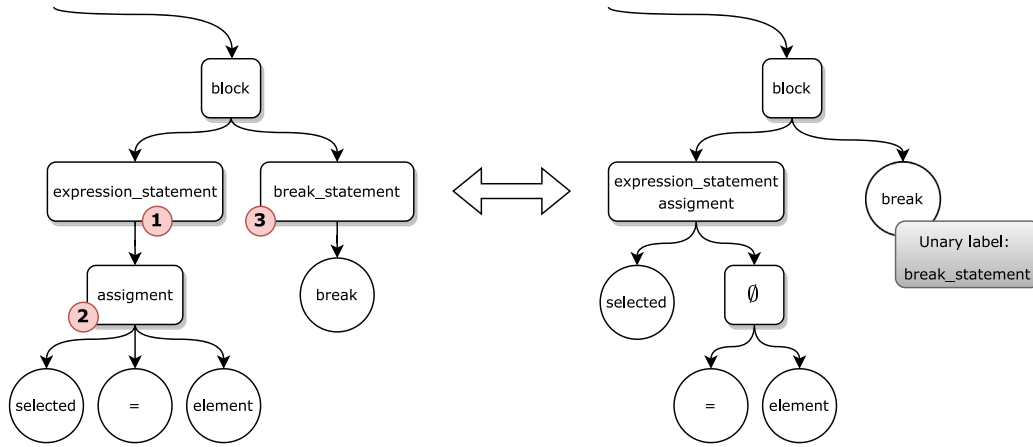


Fig. 1. Excerpt of an AST transformed into a binary tree.
Source: From López et al. (2022).

2.2.1. Probing in NLP

The probing framework has been widely used in NLP to evaluate the encoding of linguistic properties by PLMs. For example, Belinkov et al. (2017) implemented a probe to perform POS tagging over the hidden representations of a neural machine translation model. Adi et al. (2016) designed a probe on top of an auto-encoder model to predict the length of an input sentence. Probing has also been applied to evaluate whether PLMs encode more complex properties such as the syntactical structure of a language. In particular, Hewitt and Manning (2019) proposed the *Structural Probe* to investigate if dependency trees can be extracted from the hidden representations of PLMs through a linear transformation. Chi et al. (2020) extended the Structural Probe to a multilingual setting to evaluate whether the syntax is an encoded linguistic property shared cross-lingually in mBERT.

2.2.2. Probing in deep learning for code

The pioneering work in this field by Karmakar and Robbes (Karmakar and Robbes, 2021) introduces a set of four probing tasks aimed at evaluating the comprehension capabilities of PLM encoders. These tasks include cyclomatic complexity prediction, code length prediction, AST tagging, and invalid type prediction. Their approach involves applying a linear layer over the PLM output representations to determine the models' effectiveness on these tasks. This research is later expanded by the same authors, where they present the INSPECT framework (Karmakar and Robbes, 2023), an extensible probing framework comprising 15 coding tasks that examine various characteristics of source code, including surface, syntactic, structural, and semantic properties. Similarly, Troshin and Chirkova (2022) and Ma et al. (2024) introduce and evaluate additional probing tasks that examine the syntax and semantic understanding of PLMs in code contexts. Shen et al. (2022) released CodeSyntax, a large-scale dataset containing programs annotated with syntactic relationships, producing tree structures akin to dependency parsing trees in NLP. They leverage this dataset to evaluate the syntactic comprehension of code-focused PLMs. Finally, Wan et al. (2022) developed a probing method capable of extracting unlabeled binary trees from code representations.

Our work focuses on assessing the syntactic understanding of PLMs and differs from prior studies in two key ways. First, we analyze eight different programming languages, enabling us to draw more language-independent insights into PLMs' syntax comprehension and the way information is encoded across layers. Among the studies discussed, Wan et al. (2022) examined the highest number of languages, analyzing three. Second, to the best of our knowledge, this is the first study to conduct multilingual probing in the context of code, providing evidence that syntax is encoded in a multilingual manner.

3. Preliminaries on the AST-probe

In this paper, we contribute a theoretical and experimental extension to the “AST-Probe” introduced in our previous work (López et al., 2022). Therefore, we first introduce the theoretical components of the AST-Probe upon which we build our new probe to ease the reading.

As expressed in Eq. (1), a PLM learns to map a sequence of tokens into a set of hidden representations $[h_0^\ell, \dots, h_n^\ell]$, for each of its transformer layer ℓ .² Each h_i^ℓ belongs to $\mathbb{R}^{m_1}$, where m_1 is the dimension of the hidden spaces of the PLM. After a pre-training phase, the PLM is capable of embedding meaningful information about the input tokens and language into those representations. However, the exact nature of the information encoded in the hidden representations of PLMs is challenging to determine due to the inherent black-box nature of neural networks. In this context, the AST-Probe was introduced to determine if existing language models pre-trained on code implicitly learn the syntactical structure of programming languages. More specifically, the AST-Probe allows for establishing whether ASTs of codes are embedded in the geometry of the hidden representation spaces of the PLM.

The AST-Probe relies on the following hypothesis:

AST-Probe hypothesis: Given a programming language and a PLM, there exists a “syntactic subspace” within the hidden representation spaces of the PLM containing all the syntactic information of the programming language.

The objective of the AST-Probe is to find a syntactic subspace S and use the geometry inside this subspace to predict the AST of an input code snippet. The AST-Probe consists of two main components: (1) a vectorized representation equivalent to the AST, and (2) the syntactic subspace and the probing classifier.

3.1. Vectorized-equivalent representation of the AST

To enable AST prediction via classifiers, we use a classifier-friendly representation: the tuple (d, c, u) . This tuple is a vectorized equivalent of the AST for a given code snippet, allowing the prediction task to be reduced to computing just these three vectors.

From an AST to a (d, c, u) . Figs. 1 and 2 show a fragment of an AST and its mapping to this tuple. First, the AST is binarized (Fig. 1) by ① merging non-terminal unary chains, ② binarizing branches with more than two children, and ③ removing terminal chains. From this

² For sake of clarity, we remove the superscript ℓ from the hidden vectors in the next equations.

binarized AST, the tuple (d, c, u) is extracted. d is a real-valued vector that contains the height of the lowest common ancestor for every two consecutive leaves. c is a vector of categories that contains the label of the lowest common ancestor for every two consecutive leaves. Fig. 2 shows the vectors d and c extracted from the binary tree of Fig. 1. Finally, for each token, u contains the unary labels extracted in the rule ③. If a token does not have a unary label, a special category is assigned to the token, which indicates the absence of a unary label. It is clear that d encodes the structure of the tree, whereas c and u capture the non-terminal information. The labels of c are called constituency labels (Shen et al., 2018), and the labels of u are called unary labels. In practice, the constituency labels encode much more information about the non-terminals of the AST than the unary labels, as most tokens do not have a unary label.

From a (d, c, u) to an AST. Given a potentially predicted tuple (d, c, u) , the main challenge lies in reconstructing the corresponding binary AST. To address this, we employ Algorithm 1, adapted from Shen et al. (2018). Once the binary AST is obtained, converting it into a conventional AST is straightforward: this simply involves reversing the three binarization rules.

Let us denote by p_1 the procedure that transforms an AST into a tuple, and by p_2 the inverse procedure. If $p_1(t) = (d, c, u)$, then applying the inverse yields $p_2((d, c, u)) = t$. Moreover, if d' is a vector whose elements preserve the same relative ordering (i.e., ranking) as those in d (thus representing a valid syntactic distance for the binarized form of t) and $c' = c$, $u' = u$, then it also holds that $p_2((d', c', u')) = t$. Therefore, when training the classifiers to get d , we focus on approximating the rank.

Algorithm 1 (d, c, u) to binary tree function extracted from Shen et al. (2018).

```

1: function TUPLE2TREE( $d, c, u$ )
2:   if  $d == []$  then
3:      $node \leftarrow \text{Leaf}(u[0])$ 
4:   else
5:      $i \leftarrow \text{argmax}_i(d)$ 
6:      $child_l \leftarrow \text{TUPLE2TREE}(d_{<i}, c_{<i}, u_{<i})$ 
7:      $child_r \leftarrow \text{TUPLE2TREE}(d_{>i}, c_{>i}, u_{>i})$ 
8:      $node \leftarrow \text{Node}(child_l, child_r, c[i])$ 
9:   end if
10:  return node
11: end function

```

3.2. Syntactic subspace and probing classifier

Let us denote t , the ground-truth AST and (d, c, u) its corresponding tuple. The AST-Probe hypothesis states that there exists a syntactic subspace $S \subseteq \mathbb{R}^{m_1}$ that encodes the AST information.

Given the subspace S , the AST-Probe inference works as follows:

1. Each vector $h_0, \dots, h_n$ is projected onto S to obtain their corresponding syntactic vectors: $P_S(h_0), \dots, P_S(h_n) \in S$.
2. The vector $\hat{d}$ is predicted using the squared Euclidean distance between each consecutive syntactic vector: $\hat{d}_i := \|P_S(h_{i-1}) - P_S(h_i)\|_2^2$.
3. The labels $\hat{c}$ are predicted using the difference between each consecutive syntactic vector, a set of vectors $C \subset S$, and the dot product. Each $\hat{c}_i$ is the label that has the highest $\langle P_S(h_i) - P_S(h_{i+1}), v_c \rangle$ where $v_c \in C$.
4. The labels $\hat{u}$ are predicted using each syntactic vector, a set of vectors $U \subset S$, and the dot product. Each $\hat{u}_i$ is the label that has the highest $\langle P_S(h_i), v_u \rangle$ where $v_u \in U$.

To train the probing classifier, we define S by considering a matrix B of dimension $m_2 \times m_1$ whose m_2 rows define an orthonormal basis of

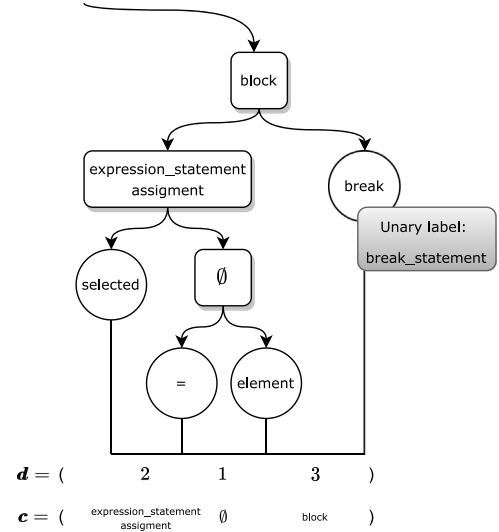


Fig. 2. d and c vectors extracted from the binary tree of Fig. 1.

S and the dimension of the syntactic subspace. That is, we can compute $P_S(h_i) = Bh_i$ and the resulting vector corresponds to the coordinates of $P_S(h_i)$ w.r.t the basis induced by B . The vectors of C, U are also defined using that basis. Altogether, the probe aims to learn the matrix B and the vectors C and U by minimizing a training loss composed of three individual loss terms and an orthogonality regularization term, which enforces the orthogonality of $P_S \leftrightarrow B$. Consequently, the optimization process jointly minimizes all three losses simultaneously:

$$L(t \parallel \hat{t}) = L_d(t \parallel \hat{t}) + L_c(t \parallel \hat{t}) + L_u(t \parallel \hat{t}) + \lambda \cdot \text{OR}(B).$$

The first loss focuses on predicting an accurate d . Since our primary goal is to match the rank order of the ground truth, we employ a pairwise learning-to-rank loss that encourages the predicted vector's components to reflect the same ranking as the ground truth.

$$L_d(t \parallel \hat{t}) = \sum_{i,j>i} \text{ReLU}(1 - \text{sign}(d_i - d_j)(\hat{d}_i - \hat{d}_j)).$$

The second loss is designed to predict the correct constituency labels. This is achieved by applying a cross-entropy loss to the dot products between each constituency vector and the corresponding projections.

$$L_c(t \parallel \hat{t}) = - \sum_i \log \frac{\exp(\langle P_S(h_i) - P_S(h_{i+1}), v_{c_i} \rangle)}{\sum_{v \in C} \exp(\langle P_S(h_i) - P_S(h_{i+1}), v \rangle)}.$$

The third loss is defined similarly using the cross-entropy loss, the unary vectors, and the projections:

$$L_u(t \parallel \hat{t}) = - \sum_i \log \frac{\exp(\langle P_S(h_i), v_{u_i} \rangle)}{\sum_{v \in U} \exp(\langle P_S(h_i), v \rangle)}.$$

Finally, the regularization term to ensure orthonormality of B is defined as follows (Limisiewicz and Mareček, 2020):

$$\text{OR}(B) = \|BB^T - I\|_F^2,$$

where $\|\cdot\|_F$ is the Frobenius norm and I the identity matrix of dimension m_2 .

4. Multilingual AST-Probe: Direct transfer & Shared subspace

The AST-Probe derives an individual syntactic subspace for each studied programming language. In other terms, the information encoded in each syntactic subspace is specific to a programming language, although there exist numerous shared characteristics among object-oriented programming languages.

In this paper, we aim to explore the overlapping or shared characteristics within these syntactic subspaces. Our hypothesis is that hidden representations in PLMs possess a cross-lingual nature, where the syntactic information of different programming languages is not entirely segregated within the model's hidden representation spaces. To examine this hypothesis, we design two key approaches: (1) direct transfer in syntactic subspaces (see Section 4.1) and (2) computation of a shared syntactic subspace (see Section 4.2). The first approach involves utilizing the syntactic subspace of one language to predict the ASTs of code snippets from another programming language. Successful prediction of these ASTs would indicate an overlapping of the two given syntactic subspaces. In the second approach, our objective is to determine whether it is feasible to derive a shared syntactic subspace that enables the prediction of ASTs for multiple programming languages. Identifying such a subspace successfully would provide additional evidence supporting the cross-lingual nature of syntactic information encoded within PLMs.

4.1. Direct transfer in syntactic subspaces

In this approach, we leverage the syntactic subspace of one language i (i.e., source language) to recover the ASTs of code snippets in another language j (i.e., target language). If we can effectively recover the ASTs in language j , we can claim that the syntactic subspace of language i also encodes some syntactic information of the language j .

Let us denote (S_i, C_i, U_i) the syntactic subspace, constituency vectors, and unary vectors of a language i , respectively. To perform direct transfer from i to j (denoted as $i \rightarrow j$), we define $(S_{i \rightarrow j}, C_{i \rightarrow j}, U_{i \rightarrow j})$ as follows:

- $S_{i \rightarrow j} := S_i$. The hidden vectors $h_0, \dots, h_n$ are projected onto the syntactic subspace of the source language i .
- $C_{i \rightarrow j} := \{P_{S_j}(v) \mid v \in C_j\}$. The constituency vectors of the target language j are projected onto the syntactic subspace of the source language.
- $U_{i \rightarrow j} := \{P_{U_j}(v) \mid v \in U_j\}$. Similarly, the unary vectors of the target language j are projected onto the syntactic subspace of the source language.

It is important to note that, by definition, we have $C_{i \rightarrow j}, U_{i \rightarrow j} \subset S_{i \rightarrow j} = S_i$. In other terms, both the constituency vectors and unary vectors of language j are projected onto the syntactic subspace of language i , and the inference process occurs exclusively within the syntactic subspace of language i . Consequently, only the syntactic subspace of language i is involved in recovering ASTs of code snippets written in language j . Furthermore, it is important to note that this approach does not involve any additional training procedures to those required by the standard AST-Probe.

To exemplify this explanation, let us assume Python as the source language and Java as the target language ($i = \text{Python}$ and $j = \text{Java}$). In the direct transfer AST-Probe, we use the syntactic subspace of Python to recover the ASTs of Java code snippets. To do this, we first project the constituency vectors of Java onto the Python subspace ($C_{i \rightarrow j}$) and similarly project the unary vectors ($U_{i \rightarrow j}$). As a result, we can recover Java code snippets by applying the standard AST-Probe with the Python syntactic subspace and the projected constituency and unary vectors.

4.2. Shared syntactic subspace

The primary objective of this approach is to investigate the existence of a shared syntactic subspace capable of encoding the complete syntax of a given set of programming languages. If successful in extracting such a shared syntactic subspace from the hidden representation spaces of a PLM, it would provide evidence that the PLM acquired cross-lingual representations of the syntactic structures of the programming languages during pre-training.

Let us consider a set $\mathcal{L}$ of programming languages. We are interested in finding a *syntactic subspace* S shared across the languages. We fix the same transformation matrix B for all languages and consider two sets of vectors $C_l, U_l \subset S$ for each $l \in \mathcal{L}$.

During the inference phase, regardless of the programming language, we project the input vectors onto the shared syntactic subspace using the transformation matrix B . We then make predictions based on the projected vectors, obtaining d . If the input code is written in language l , we use the vectors C_l and U_l to predict c and u , respectively. In this case, the categories of the other languages are masked or ignored.

When training the probe and to achieve optimality of the shared syntactic subspace S , we modify the original loss function to support the described multilingual setting:

$$\begin{aligned} L(t \parallel t') &= L_d(t \parallel t') \\ &+ \sum_{l \in \mathcal{L}} L_{c,l}(t \parallel t') I(l, t') \\ &+ \sum_{l \in \mathcal{L}} L_{u,l}(t \parallel t') I(l, t') \\ &+ \lambda \cdot \text{OR}(B), \end{aligned}$$

where $I(l, t')$ is an indicator function that establishes whether the ground-truth tree belongs to the language l , $L_{c,l}(t \parallel t')$ is the loss for the constituency labels of the language l , and $L_{u,l}(t \parallel t')$ is the loss for the unary labels of the language l .

In our scenario, we consider a pool of eight programming languages: Python, Java, Ruby, JavaScript, Go, PHP, C, and C#. In this context, the approach consists of concatenating the training examples from these three languages and train the standard AST-Probe. To ensure that the gradients of the constituency/unary vectors for one programming language do not interfere with those of another, we mask the constituent/unary labels that are not relevant to the input code snippet.

5. Experimental setup

In this section, we present a comprehensive overview of our experimental setup, detailing the data and models employed. Our main objective closely aligns with that of Chi et al. (2020): has CodeBERT learned a cross-lingual syntactic representation? However, our focus differs in that we examine programming languages rather than natural languages. In this context, syntax is typically represented using ASTs, rather than simpler dependency structures. Consequently, the widely used structural probe (Hewitt and Manning, 2019) is not applicable, which led us to adopt our previously proposed multilingual AST probe, specifically adapted for this purpose. Similarly to Chi et al. we organize our study around four main research questions. The first three build on the findings of our previous work (López et al., 2022) by applying the monolingual AST-Probe across a broader range of programming languages and PLMs, thereby expanding the scope of the analysis. This monolingual investigation serves as a foundational step for subsequent multilingual analysis, as demonstrating the existence of monolingual syntactic subspaces within a PLM is a prerequisite for identifying a shared syntactic subspace (Chi et al., 2020). The final research question introduces the multilingual AST-Probe approach, aiming to determine whether CodeBERT has learned a cross-lingual syntactic representation.

- **RQ1: How well do PLMs understand the syntax of programming languages?** We assess the syntactic understanding of eight PLMs across eight programming languages and compare their performance against a strong baseline. A PLM surpassing the baseline indicates a substantial understanding of the syntax of the studied programming languages.
- **RQ2: From which PLM can the ASTs be best reconstructed?** Using the AST-Probe, we extract ASTs for eight programming languages from the hidden representations of eight different PLMs and evaluate their quality against ground truth ASTs. We then identify which PLM most effectively captures AST structures in its hidden representations.

- **RQ3: How is syntactic information encoded across different layers of the PLMs?** We apply the AST-Probe to hidden representation spaces of intermediate layers in the studied PLMs. By comparing the effectiveness of the probe across different layers, we analyze how the syntactic information is distributed within the hidden representation spaces of each PLM.
- **RQ4: Does CodeBERT learn a cross-lingual representation of the syntactic structure?** We select CodeBERT as a representative PLM and investigate whether it acquires a cross-lingual representation of the syntax of programming languages during its pre-training phase. To examine this aspect, we experiment using the two novel adaptations of the AST-Probe introduced in the previous section: direct transfer and shared syntactic subspace.

5.1. Data and models

Datasets. We experiment with the following programming languages: Python, Java, Ruby, Go, JavaScript, PHP, C, and C#. We considered CodeSearchNet (Husain et al., 2019) dataset for Python, Java, Ruby, Go, JavaScript and PHP. To collect snippets of C, we picked the Devign dataset introduced in Zhou et al. (2019). For C#, we use CodeTrans dataset (C# to Java) of CodeXGlue (Lu et al., 2021). To train the probes, we sample a maximum of 20,000 code snippets for each language and split them into training/test/validation sets following a 70/20/10 ratio. We do not need many samples to train the probes as they are parameter-efficient and converge fast using a limited number of training samples. Finally, to construct the ASTs of the code snippets, we rely on the tree-sitter compiler tool,³ which allows us to efficiently parse all the studied programming languages.

Models. We considered eight PLMs. Four of them were pre-trained using natural language: BERT (Devlin et al., 2018), DistilBERT (Sanh et al., 2019), RoBERTa (Liu et al., 2019) and DistilRoBERTa (Wolf et al., 2019). The remaining were either trained on code or bimodal data (code and natural language documentation): CodeBERTa (Wolf et al., 2019), CodeT5⁴ (Wang et al., 2021), CodeBERT (Feng et al., 2020) and GraphCodeBERT (Guo et al., 2020). All these code models were pre-trained using the CodeSearchNet dataset (Husain et al., 2019), which comprises code snippets written in Python, Java, Ruby, Go, JavaScript, and PHP. Additionally, CodeT5 incorporates a dataset containing C and C# snippets extracted from BigQuery.

Baseline. We use CodeBERTrand as a baseline. The baseline's embedding layer is initialized with CodeBERT's pre-trained weights, while its transformer layers are randomly initialized. This baseline has been established as a strong baseline in NLP probing studies (Chi et al., 2020; Hewitt and Manning, 2019). The objective of the baseline is to verify that the information extracted from the PLMs through probing cannot be retrieved from a randomly initialized model. Therefore, if a PLM performs better in the probe than the baseline, it suggests that the capacity to recover the information is an inherent property of the PLM.

5.2. Evaluation metrics

The output of the AST-Probe is a prediction for the vectors d , c and u . To get a meaningful interpretation of the effectiveness of the models, we compare the predicted AST, i.e., recovered from the vectors d , c and u , with the ground-truth AST. To this end, we compute the precision, recall and F_1 score over tree constituents (Abney et al., 1991). These three metrics are used in the NLP literature to evaluate constituency parsers and are defined as:

$$\text{Prec} = \frac{|\text{Const. in prediction}| \cap |\text{Const. in ground-truth}|}{|\text{Const. in prediction}|}$$

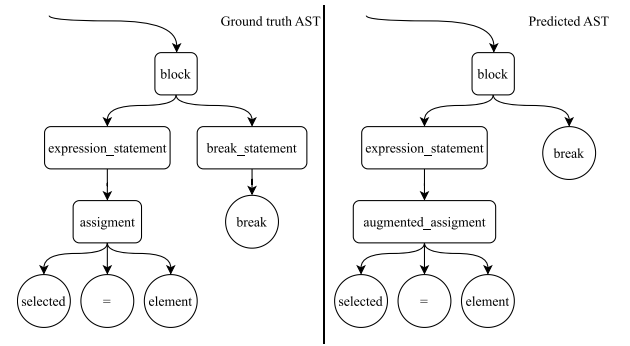


Fig. 3. Excerpt of a predicted AST. (a) Original AST. (b) Predicted AST with some errors (i.e., augmented_assignment and break).

$$\text{Recall} = \frac{|\text{Const. in prediction}| \cap |\text{Const. in ground-truth}|}{|\text{Const. in ground-truth}|}$$

$$F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Recall}}{\text{Prec} + \text{Recall}}$$

In the context of ASTs, we define the constituents as (non-terminal, *scope*) where *scope* indicates the position of the first and last tokens. There is one constituent for each non-terminal node. For instance, the constituents of the AST of Fig. 3 (left) are the following:

1. (block, from *selected* to *break*)
2. (expression_statement, from *selected* to *element*)
3. (assignment, from *selected* to *element*)
4. (break_statement, from *break* to *break*)

Let us assume that our predicted AST with respect to that block is the one of Fig. 3 (right). Thus, its constituents are the following (✓ represents a hit and ✗ a miss with respect to the true AST):

1. (block, from *selected* to *break*) ✓
2. (expression_statement, from *selected* to *element*) ✓
3. (augmented_assignment, from *selected* to *element*) ✗

In that case, the precision and recall are 2/3 and 2/4, respectively.

5.3. Addressing the research questions

To answer RQ1/2/3, we train the AST-Probe for each combination of programming language, model, and layer, resulting in the training and comparison of 720 probes.⁵ For all RQs, we report the F_1 score as the evaluation metric.

To answer RQ4, we select the best syntactic layer of CodeBERT and run our extensions of the AST-Probe. Particularly, for each pair of languages (i, j) where $i \neq j$, we compute the direct transfer probing $i \rightarrow j$ and report the F_1 score. Finally, we extract the shared syntactic subspace and report its performance when recovering the AST of all studied programming languages.

5.4. Training details

All the considered models were pre-trained using *byte-pair encoding* (BPE) (Sennrich et al., 2015) which constructs the vocabulary of the model over subwords. Since our analysis is performed over whole-word tokens, we assign to each token representation the average of its subword representations (Hewitt and Manning, 2019; Wan et al., 2022). For all models, the dimension of the word embedding space is

³ <https://tree-sitter.github.io/tree-sitter/>

⁴ CodeT5 is an encoder-decoder. We only consider the transformer layers of the encoder.

⁵ $720 = (12 \text{ layers} \times 6 \text{ big models} \times 8 \text{ PLMs}) + (6 \text{ layers} \times 3 \text{ small models} \times 8 \text{ PLMs})$.

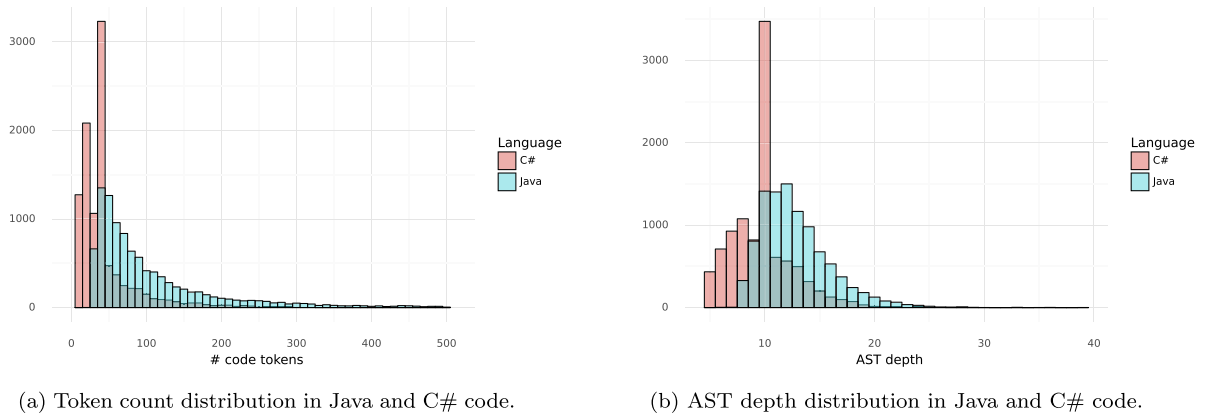


Fig. 4. Comparison of token count and AST depth distributions in Java and C# code.

$m_1 = 768$. We set the orthogonal regularization term to a high value $\lambda = 5$ in order to ensure orthogonality of the basis induced by the matrix B . Finally, we set the dimension of the syntactic subspace to 128, as it was the estimated optimal dimension in our previous work (López et al., 2022).

To perform the optimization, we use the Adam optimizer (Kingma and Ba, 2014) with an initial learning rate of 0.001. After each epoch, we reduce the learning rate by multiplying it by 0.1 if the validation loss does not decrease. We set the maximum number of epochs to 20 and use early-stopping with a patience of five epochs. A similar configuration of hyperparameters is used in Hewitt and Manning (2019) to train the syntax probe.

6. Results

6.1. RQ1: Syntax understanding of PLMs

Table 1 presents the performance of the AST-Probe for each PLM and programming language. The representative layer for each PLM is selected based on the highest average F_1 score achieved across all languages.

The results reveal a significant performance gap between the baseline and the PLMs across all programming languages. This finding further validates the effectiveness of the AST-Probe, as its evaluation has been extended to a wider range of PLMs and programming languages compared to our previous work (López et al., 2022). Moreover, the inability of the AST-Probe to generate ASTs from non-informative code representations highlights that all PLMs, including those pre-trained on natural language data, possess a certain degree of understanding of code syntax.

The F_1 scores for the C# programming language are the highest. This can be attributed to the fact that the C# code snippets are sourced from the CodeTrans dataset, which predominantly contains code with simple syntax and largely consists of basic API calls or standard getter and setter methods. Figs. 4(a) and 4(b) show the distributions of code token counts and AST depths for a sample of 10,000 C# snippets (from CodeTrans) and 10,000 Java snippets (from the CodeSearchNet dataset). We select the Java dataset for comparison with C# because the two languages share significant syntactic similarities, including many common constructs and keywords. The C# snippets are noticeably simpler, with lower token counts (45.41 ± 37.56 vs. 105.07 ± 86.86) and AST depths (9.74 ± 2.71 vs. 12.78 ± 3.20). For illustration, Listing 1 presents two randomly selected C# examples, both featuring straightforward syntax with just one or two assignment statements.

Answer to RQ1: All PLMs consistently outperform the baseline by a substantial margin. This substantial performance gap provides compelling evidence that these models acquired a substantial amount of syntactic knowledge about code during their pre-training phase.

```
public virtual void SetOffset(long offset){
    this.offset = offset;
}

public CreateVaultRequest(string accountId, string
    vaultName){
    _accountId = accountId;
    _vaultName = vaultName;
}
```

Listing 1: Two random samples from the C# dataset.

6.2. RQ2: Best PLM

Based on the findings in Table 1, we highlight two key observations. Firstly, both CodeBERT and GraphCodeBERT consistently outperform the other PLMs across all tested programming languages. While GraphCodeBERT shows a slightly higher average performance, the difference is not statistically significant (p -value = 0.7422⁶).

Secondly, despite being trained on textual data rather than code-specific data, RoBERTa demonstrates competitive performance. Remarkably, RoBERTa achieves results similar to CodeBERTa (p -value = 0.3125), a model specifically trained on code data. This suggests that RoBERTa possesses a comparable understanding of code syntax to that of code-specific models, reinforcing its reputation as a strong baseline for code-related tasks (Lu et al., 2021).

Answer to RQ2: Among all PLMs, both CodeBERT and GraphCodeBERT demonstrate a superior capability of encoding ASTs in their hidden representations.

6.3. RQ3: Syntactic information across layers

Fig. 5 presents an analysis of the F_1 score evolution across intermediate layers for GraphCodeBERT, for each programming language. This plot shows that the distribution of syntactic information across layers is independent of the programming language. In fact, it is clear that all the lines follow the same trend, and the layers that perform the best are roughly the same across languages. This finding indicates that the syntactic information encoded in the different layers of the models is not specific to individual programming languages and follows a similar

⁶ This p -value was calculated using the paired Wilcoxon test on the rows for CodeBERT and GraphCodeBERT in Table 1.

Table 1

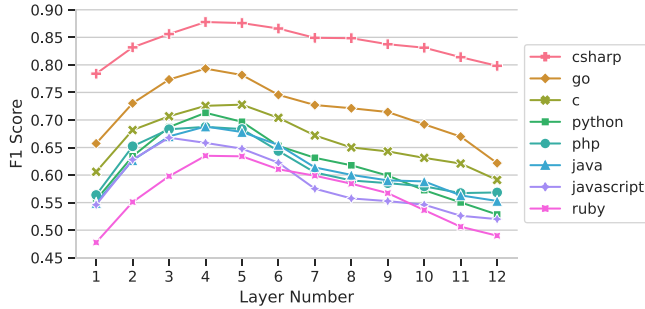
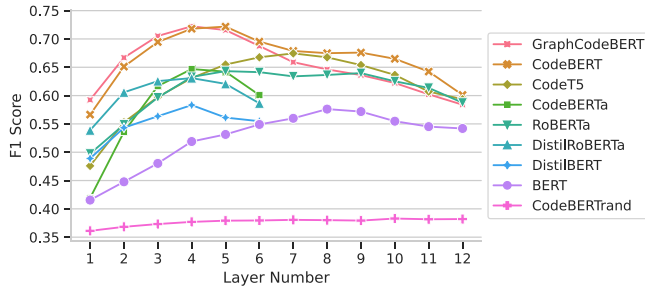
AST-Probe results (F_1 score). The subscripts *NL* and *code* denote whether the PLM is a natural language model or a code model, respectively. Δ_{baseline} indicates the model's improvement relative to the baseline.

MODEL-#LAYER	Python	Java	Ruby	JavaScript	Go	PHP	C	C#	Average
CodeBERTrand-10	0.3231	0.3112	0.2843	0.3205	0.4021	0.3922	0.3741	0.6556	0.3829
BERT-8 _{NL}	0.5324	0.5144	0.4486	0.5306	0.6415	0.5789	0.5831	0.7796	0.5761
Δ_{baseline}	0.2093	0.2032	0.1643	0.2101	0.2394	0.1867	0.2090	0.1240	0.1932
DistilBERT-4 _{NL}	0.5446	0.5377	0.4488	0.5260	0.6393	0.5802	0.5974	0.7919	0.5832
Δ_{baseline}	0.2215	0.2265	0.1645	0.2055	0.2372	0.1880	0.2233	0.1363	0.2003
RoBERTa-5 _{NL}	0.6124	0.5849	0.5278	0.5813	0.7181	0.6222	0.6665	0.8321	0.6432
Δ_{baseline}	0.2893	0.2737	0.2435	0.2608	0.3160	0.2300	0.2924	0.1765	0.2603
DistilRoBERTa-4 _{NL}	0.6112	0.5719	0.5149	0.5780	0.7093	0.6103	0.6418	0.8096	0.6309
Δ_{baseline}	0.2881	0.2607	0.2306	0.2575	0.3072	0.2181	0.2677	0.1540	0.2480
CodeBERTa-4 _{code}	0.6111	0.5936	0.5245	0.5892	0.7225	0.6410	0.6601	0.8329	0.6469
Δ_{baseline}	0.2880	0.2824	0.2402	0.2687	0.3204	0.2488	0.2860	0.1773	0.2640
CodeT5-7 _{code}	0.6430	0.6123	0.5942	0.6103	0.7436	0.6705	0.6707	0.8515	0.6745
Δ_{baseline}	0.3199	0.3011	0.3099	0.2898	0.3415	0.2783	0.2966	0.1959	0.2916
CodeBERT-5 _{code}	<u>0.7047</u>	<u>0.6810</u>	<u>0.6211</u>	0.6683	<u>0.7832</u>	0.7003	0.7320	0.8839	<u>0.7218</u>
Δ_{baseline}	0.3816	0.3698	0.3368	0.3478	0.3811	0.3081	0.3579	0.2283	0.3389
GraphCodeBERT-4 _{code}	0.7130	0.6881	0.6350	0.6584	0.7932	0.6875	0.7257	0.8778	0.7223
Δ_{baseline}	0.3899	0.3769	0.3507	0.3379	0.3911	0.2953	0.3516	0.2222	0.3394

Table 2

Statistic W for each PLM. The subscripts *NL* and *code* denote whether the PLM is a natural language model or a code model, respectively.

Models	GraphCodeBERT _{code}	CodeBERT _{code}	CodeT5 _{code}	CodeBERTa _{code}	DistilRoBERTa _{NL}	RoBERTa _{NL}	DistilBERT _{NL}	BERT _{NL}
W	0.9541	0.9231	0.9386	0.9786	0.8179	0.8431	0.8429	0.9451

**Fig. 5.** F_1 score for each language and layer of GraphCodeBERT.**Fig. 6.** Average F_1 score for each PLM and layer.

distribution for each language. To statistically verify this observation, we compute Kendall's W coefficient to compare the ranks induced by each language in the layers. The coefficients for each PLM are presented in Table 2. Notably, we observed a strong agreement among all the models.

Finally, Fig. 6 depicts the F_1 score for each model across layers, averaged over all programming languages. We can observe that the syntactic information is encoded in the middle layers for all PLMs.

Answer to RQ3: For all models, the syntactic information distribution across layers is independent of the programming language. Furthermore, the syntactic information is mostly encoded in the middle layers of the PLMs.

6.4. RQ4: Cross-lingual probing on CodeBERT

A key finding from the previous research questions is the presence of monolingual syntactic subspaces in several PLMs, particularly in CodeBERT. This serves as a crucial foundation for establishing a shared syntactic understanding (Chi et al., 2020). In this RQ, we provide evidence that CodeBERT has acquired a shared syntactic understanding across programming languages through the two multilingual AST-Probe extensions. Although all other models may exhibit some degree of multilinguality in terms of syntax, conducting all cross-lingual experiments (both direct transfer and shared subspace) across every model would require extensive training and significant GPU resources. We choose CodeBERT as the representative for multilingual experiments because (1) it is one of the most widely used state-of-the-art encoders, and (2) along with GraphCodeBERT, their hidden spaces offer the most effective encoding of ASTs.

For each pair of programming languages i, j , we show the performance of the direct transfer probing in Table 3. It is important to note that surpassing the baseline in this context is challenging due to three main factors: (1) the baseline's AST-Probe is strong, having been trained exclusively on samples from the target language; (2) there is a significant distribution shift between code in different languages; and (3) the grammar definitions of programming languages can differ substantially. We elaborate on these three points further in Section 9.4.

We can observe that, in five out of eight programming languages, there is at least one case that outperforms the baseline. This indicates that the language-specific syntactic subspaces within CodeBERT overlap to some extent. Notably, the syntactic subspace of JavaScript demonstrates the highest transferability, as it enables the recovery of ASTs for multiple languages (Python, Java, Ruby, and C) better than the baseline.

Finally, we compute the shared syntactic subspace by concatenating all the training samples from all languages. Table 4 presents the F_1

Table 3

Direct transfer probing performance for each pair of languages. Bold scores outperform the monolingual baseline.

$i \rightarrow j$	Python	Java	Ruby	Javascript	Go	C	C#	PHP
Baseline	0.3231	0.3112	0.2843	0.3205	0.4021	0.3741	0.6556	0.3922
Python	–	0.2460	0.3648	0.3995	0.3478	0.2765	0.3829	0.1563
Java	0.2150	–	0.2250	0.378	0.1673	0.3575	0.5925	0.1889
Ruby	0.3709	0.1823	–	0.2928	0.2288	0.2134	0.3001	0.1556
Javascript	0.4303	0.3467	0.3198	–	0.3481	0.3959	0.4635	0.1334
Go	0.3946	0.2705	0.2465	0.3909	–	0.3018	0.3841	0.1562
C	0.2367	0.2386	0.1721	0.3278	0.2133	–	0.4488	0.1693
C#	0.2291	0.3581	0.1910	0.3585	0.2556	0.3476	–	0.1880
PHP	0.2113	0.2621	0.1397	0.2486	0.2176	0.1770	0.4091	–

Table 4

Shared subspace performance for each language.

Models/Languages	Python	Java	Ruby	Javascript	Go	C	C#	PHP
Baseline	0.3231	0.3112	0.2843	0.3205	0.4021	0.3741	0.6556	0.3922
Shared subspace	0.6454	0.6292	0.5675	0.6262	0.7178	0.689	0.8293	0.6629

scores for the shared syntactic subspace on the test set of each language. We observe that, in all languages, the baseline is significantly outperformed.

Answer to RQ4: We found evidence that CodeBERT learns a cross-lingual representation of the syntactic structure of code. ASTs for five out of eight languages can be recovered by leveraging the syntactic subspaces of another language. Additionally, there exists a shared syntactic subspace that captures the syntax of all the languages.

7. Case study

This section presents a case study illustrating the monolingual and multilingual probes. Listing 2 provides the Python code used in this case study, while Fig. 7 depicts its corresponding AST. For this analysis, we focus on CodeBERT as the target model, since we have computed both the multilingual syntactic subspace and the monolingual ones for this model in our experiments. Additionally, we examine the hidden representation space induced by layer 5 as it is the best performing syntactic layer.

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
```

Listing 2: Case study Python code snippet.

7.1. Monolingual probing

The tokenized code snippet is passed through CodeBERT to extract the hidden representations. Using the Python syntactic subspace, along with the constituency and unary vectors derived from the training set, we predict the tuple (d, c, u) . We then reconstruct it using the recursive Algorithm 1.

Once the binarized AST is obtained, generating the final predicted AST is straightforward. The process involves expanding non-terminal unary chains, removing special $\emptyset$ binarized non-terminals, and expanding unary labels, following the three mapping rules outlined in Section 3 and illustrated in Fig. 1.

For this case study, the predicted AST is shown in Fig. 8. Only one non-terminal, for `statement`, is incorrectly predicted, as it incorrectly includes a `return statement` as a child. This results in a precision of $8/9 = 0.89$. As for recall, the predicted AST omits one

block non-terminal that should group the assignment `total = 0`, the for-loop, and the return statement, yielding a recall of $8/10 = 0.8$. Overall, the quality of the predicted AST is high, with an F_1 score of 0.84.

7.2. Direct transfer

Given the strong transferability observed from JavaScript to Python in our experiments, we use the JavaScript syntactic subspace to predict an AST. Specifically, the syntactic subspace is derived from JavaScript code snippets, while the constituency and unary vectors are taken from Python but projected into the JavaScript subspace. With this setup, once the tuple (d, c, u) is obtained, the process of reconstructing the AST follows the same procedure as in the monolingual probing setting.

The resulting AST, predicted using the JavaScript syntactic subspace, is shown in Fig. 9. In this case, the AST quality is lower, with a precision of $7/11 = 0.64$, recall of $7/10 = 0.7$, and an F_1 score of 0.67. The JavaScript subspace performs well in recovering simple non-terminals near the leaves (such as parameters, return statements, and basic statements) but struggles with more complex structural elements.

7.3. The shared subspace

In this case, the (d, c, u) vectors are obtained using the shared syntactic subspace, as well as the constituency and unary vectors. Those three elements were computed by training with a concatenated dataset of code snippets across all languages. The predicted AST using this shared subspace is identical to the one produced by the monolingual probing, shown in Fig. 8.

8. Threats to validity

8.1. Threats to external validity

We identified three main external threats to the validity of our experiments. The first one relates to the representativeness of our datasets to train and evaluate the probes. To mitigate this threat, we choose datasets well-known in the literature (Husain et al., 2019; Lu et al., 2021; Zhou et al., 2019) to cover eight programming languages. Furthermore, we select random samples from those datasets to train the AST-Probe and its extensions. The second external threat relates to the choice of the PLMs. To mitigate this threat, we select a broad pool of PLMs of code to study their understanding of programming languages' syntax. Additionally, we cover PLMs from the NLP literature to provide more insights about the probes. The third aspect concerns the generalizability of the method to decoder-only models, e.g., large

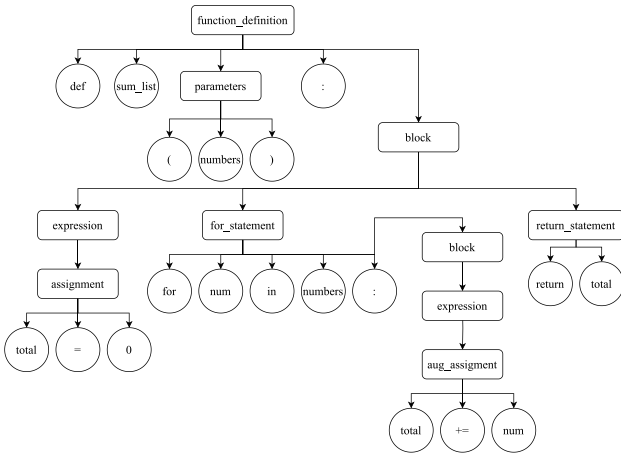


Fig. 7. Gold AST of the case study snippet.

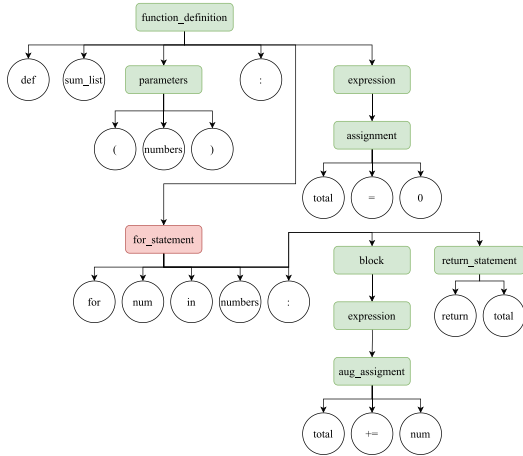


Fig. 8. Predicted AST using CodeBERT's monolingual subspace. Green non-terminal nodes indicate matches with the gold AST, while red nodes indicate mismatches. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

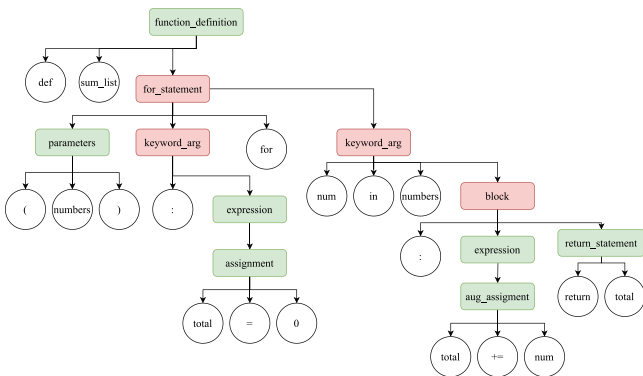


Fig. 9. Predicted AST using the JavaScript subspace. Green non-terminal nodes indicate matches with the gold AST, while red nodes indicate mismatches. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

language models (LLMs). To mitigate this threat, we conducted preliminary experiments with Qwen2.5-Coder-0.5B (Hui et al., 2024), Qwen2.5-Coder-0.5B-Instruct (Hui et al., 2024), NextCoder-7B (Aggarwal et al., 2025) using our multilingual AST-Probe. Table 5 reports the

F_1 scores for the shared syntactic subspace on all test sets for these models compared to CodeBERT. We observe that CodeBERT substantially outperforms all three decoder-based models across languages, regardless of training approaches (base code model vs instruction-tuned on code vs fine-tuning for code edits generation). This suggests that the performance difference is more likely attributable to architectural differences rather than model scale or training methodology. The fact that NextCoder-7B still underperforms CodeBERT indicates that the observed differences are not simply due to model capacity limitations. This supports our hypothesis that the autoregressive pre-training objective and causal attention mechanisms in decoder-only models may be less suited for bidirectional syntactic understanding tasks. These results align with prior findings in Karmakar and Robbes's paper (Karmakar and Robbes, 2023), where the authors also observed the poor performance of decoder-only models in probing tasks. The consistent underperformance across different decoder architectures suggests that the probing methodology might need fundamental adaptations for autoregressive models, rather than simple parameter adjustments.

8.2. Threats to internal validity

One internal threat to validity in assessing the syntax understanding of PLMs (RQ1 and RQ2) arises from reporting only the F_1 score of the best-performing layer for each PLM. These top layers represent where the PLM demonstrates the strongest understanding of programming language syntax. We focus on the best layer because it most accurately reflects the extent of the PLM's syntactic comprehension. Another internal threat to validity stems from the use of CodeSearchNet as the evaluation dataset for the probe. Some PLMs, such as CodeBERT, were pre-trained on this dataset, which could lead to overly optimistic results. However, this threat is mitigated by the fact that the encoder-only models examined in this study were pre-trained on tasks that differ significantly from the specific probing task we investigate. For instance, CodeBERT was pre-trained using Masked Language Modeling (MLM) and Replaced Token Detection (Feng et al., 2020), while GraphCodeBERT combined MLM with tasks based on control flow graphs (Guo et al., 2020). Moreover, we have evaluated code datasets in languages different from those used during PLM pre-training (C and C#) and observed consistent trends with respect to the CodeSearchNet languages: a performance gap relative to the baseline, the same ranking of PLMs, and a similar syntactic distribution across layers.

8.3. Threats to construct validity

The selection of the metric used to evaluate PLMs on the probes poses a risk to construct validity. In this paper, we observe a high similarity between ASTs and constituency trees, leading us to select the F_1 score based on tree constituents. This choice is grounded in the fact that F_1 is a well-established metric in NLP for assessing the quality of recovered trees (Mrini et al., 2019; Tian et al., 2020; Zhou and Zhao, 2019; Marcus et al., 1993). Finally, it is important to note that this metric may be influenced by both granularity and label bias (Manning and Schütze, 1999). Granularity bias arises from the tendency to favor smaller constituents, as simpler structures are easier to predict correctly, whereas larger and more complex structures are more challenging. However, the F_1 score treats all constituents equally, without assigning greater importance to more complex ones. Label bias refers to the unbalanced distribution of constituent types within code. Some constructs (e.g., variable declarations) appear more frequently than others (e.g., for-loops). The F_1 score does not account for this imbalance either. To address these limitations, it would be necessary to identify the specific grammar-level constructs for each grammar (e.g., loops, conditionals) and perform a targeted subtree analysis for each one. We leave this promising direction for future work.

Table 5

Shared subspace performance for CodeBERT, Qwen2.5-Coder-0.5B, Qwen2.5-Coder-0.5B-Instruct, and NextCoder-7B.

Models/Languages	Python	Java	Ruby	Javascript	Go	C	C#	PHP
CodeBERT	0.6454	0.6292	0.5675	0.6262	0.7178	0.6890	0.8293	0.6629
Qwen2.5-Coder-0.5B	0.5599	0.5722	0.4501	0.5175	0.6343	0.6165	0.7655	0.5998
Qwen2.5-Coder-0.5B-Instruct	0.5527	0.5565	0.4584	0.5223	0.6346	0.6111	0.7773	0.5966
NextCoder-7B	0.5694	0.5739	0.4976	0.5549	0.6603	0.6435	0.8019	0.6332

9. Discussion

In this section, we discuss key findings from our experiments, comparing them with results from previous studies.

9.1. CodeBERT vs GraphCodeBERT

Our analysis indicates that CodeBERT and GraphCodeBERT show a stronger understanding of programming language syntax compared to other encoders. However, we do not find significant statistical differences between them. One possible reason for this similarity is that GraphCodeBERT was initialized with CodeBERT and trained on the same dataset for additional epochs, making the two models closely related. While GraphCodeBERT includes data flow information during pre-training, this information is not used when training the probes, meaning GraphCodeBERT cannot utilize it in our analysis. We choose to exclude this information to ensure a fair comparison with other PLMs. Notably, in the original paper (Guo et al., 2020), GraphCodeBERT's improvement over CodeBERT in downstream tasks is marginal, as noted by one of the anonymous reviewers on OpenReview.⁷ This may further explain why both models show similar levels of syntax understanding.

9.2. The syntactic layers

Our findings regarding the best layers align with the results of Troshin and Chirkova. Troshin and Chirkova (2022) and Wan et al. (2022), who applied various syntactic probes to similar models. In particular, Troshin and Chirkova tackled a complex and closely related task — predicting the AST depth of a given code snippet — and observed a layer-wise performance distribution for CodeBERT and GraphCodeBERT that closely matches our own. Similarly, Wan et al. trained a probe that predicts distances between tokens within the AST, a task conceptually similar to the structural probe (Hewitt and Manning, 2019), using CodeBERT and GraphCodeBERT across Java, PHP, and Python. Once again, their results show a performance distribution across layers that closely resembles ours.

In NLP, several researchers have demonstrated that BERT (and BERT-like models) encode syntactic knowledge of natural languages primarily in their middle layers (Rogers et al., 2020), which aligns with our findings for programming languages. Notably, Hewitt and Manning (Hewitt and Manning, 2019) introduced the structural probe — a method for extracting dependency trees (simpler syntactic structures than ASTs used in natural language) — and found that the highest-quality trees were extracted from the middle layers of both BERT and ELMo. Interestingly, in this paper, we have similarly evaluated how BERT encodes the AST structure of source code across its layers, and the performance distribution closely mirrors the one reported by Hewitt and Manning for dependency trees. This may suggest that BERT processes the syntax of programming languages in a manner similar to how it processes the syntax of natural languages.

9.3. Layer-wise syntactic knowledge is language-independent

We find evidence that the distribution of syntax across layers within a given PLM remains consistent regardless of the programming language, as indicated by a high Kendall's W agreement across all models (see Table 4 and Fig. 5 for the GraphCodeBERT's case). This interesting finding aligns with previous findings in code probing research. For example, Wan et al. (2022) identified a similar layer-wise pattern in GraphCodeBERT and CodeBERT for Java, PHP, and Python. Similarly, in the NLP domain, Chi et al. (2020) observed this layer-wise pattern across multiple natural languages in multilingual BERT.

9.4. Challenges of the direct transfer probe

At first glance, the results obtained in RQ4 regarding the direct transfer probe might not seem remarkable, as only the ASTs of five programming languages can be recovered from others. However, these results are significant because outperforming the baseline (CodeBERT-Transd) with this type of probe is inherently challenging. This difficulty arises due to three main reasons. Firstly, the baseline's AST-Probe is strong, having been trained exclusively on samples from the target language. Furthermore, previous studies have demonstrated that such baselines are effective for testing probes (Hewitt and Manning, 2019; Wan et al., 2022). Secondly, there may be substantial distributional shifts between code samples in the source and target programming languages. For instance, some syntactic constructs are more prevalent in one language than another, and the syntax styles between languages can differ considerably. Finally, the AST-Probe depends on a grammar definition, which can vary significantly across different programming languages. In this paper, we use open-source grammar definitions for the eight programming languages, which were developed by different contributors and may not follow consistent patterns in their construction.

9.5. The cross-lingual syntactic representation of CodeBERT

In this paper, we enhance our original AST-Probe by introducing two multilingual approaches: the conservative direct transfer AST-Probe and the shared syntactic subspace. We apply these probes to demonstrate that syntactic information in CodeBERT is not stored independently for each programming language but rather in a shared manner. This unified representation of syntax provides an explanation (along with other factors like identifier naming patterns (Ahmed and Devanbu, 2022)) to the fact that these models generalize effectively to unseen programming languages (Guo et al., 2020; Feng et al., 2020; Ahmed and Devanbu, 2022). This also supports previous results showing that multilingual fine-tuning often outperforms monolingual fine-tuning (Ahmed and Devanbu, 2022).

9.6. Programming language paradigms and dataset limitations

The programming languages examined in this study predominantly follow imperative and object-oriented paradigms, with some support for procedural programming. We did not include functional programming languages (e.g., Haskell, Lisp, ML) or declarative languages (e.g., Prolog, SQL), which have fundamentally different syntactic structures and computational models. This limitation stems partly from the

⁷ <https://openreview.net/forum?id=jLoC4ez43PZ¬elid=2KLdo-CCc1p>

scarcity of large-scale, high-quality datasets for functional and declarative programming languages. The CodeSearchNet dataset (Husain et al., 2019), which forms the foundation of many code models included in this study, primarily contains imperative and object-oriented languages. Similarly, other major code datasets have historically underrepresented functional and declarative paradigms due to their lower prevalence in open-source repositories. Therefore, the “universal grammar” for code that we observe might be more accurately characterized as a grammar for imperative-style and object-oriented programming paradigms. As datasets for functional and declarative languages become more available, extending our multilingual AST-Probe to these paradigms would provide crucial insights into whether PLMs develop truly universal syntactic representations or paradigm-specific ones. This investigation could reveal whether the cross-lingual transfer we observe is due to shared code structures or represents deeper computational abstractions.

10. Conclusion and future work

In this paper, we have extended our previous AST-Probe by incorporating eight programming languages and eight encoders while adapting the AST-Probe to a multilingual setting. Our findings demonstrate that the distribution of syntactic knowledge across layers is independent of the programming language, and we provide evidence that CodeBERT has learned a cross-lingual representation of syntactic information for programming languages. For future work, our evaluation of three decoder-based models (ranging from 0.5B to 7B parameters) reveals consistent architectural limitations for syntactic probing tasks. Future research should focus on: (1) developing decoder-specific probing methodologies that leverage causal attention patterns, (2) investigating whether larger decoder models (10B+ parameters) can overcome these limitations through scale, and (3) examining whether the syntactic understanding captured by our probe is fundamentally different from the syntactic knowledge required for code generation tasks where decoder models excel.

We also aim to investigate other programming languages and paradigms, particularly functional programming languages (e.g., Haskell, Lisp) and declarative languages (e.g., Prolog, SQL), to broaden our understanding of code models and determine whether cross-lingual syntactic representations extend beyond imperative and object-oriented programming paradigms.

CRediT authorship contribution statement

José Antonio Hernández López: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Martin Weyssow:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation. **Jesús Sánchez Cuadrado:** Writing – review & editing, Writing – original draft, Supervision, Funding acquisition. **Houari Sahraoui:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Work supported by projects TED2021-129381B-C22 (MCIN/AEI/10.13039/501100011033 and NextGenEU/PRTR), PID2022-140109 NB-I00 (MCIN/AEI/10.13039/501100011033 and FEDER/UE) and CNS2022-135578 (MICIU/AEI/10.13039/501100011033 and NextGenEU/PRTR).

Data availability

All experimental data and code used in this paper are available at <https://github.com/Antolin1/multilingual-ast-probe>.

References

- Abney, S., Flickenger, S., Gdaniec, C., Grishman, C., Harrison, P., Hindle, D., Ingria, R., Jelinek, F., Klavans, J., Liberman, M., et al., 1991. Procedure for quantitatively comparing the syntactic coverage of english grammars. In: *Proceedings of the Workshop on Speech and Natural Language*. pp. 306–311.
- Adi, Y., Kermany, E., Belinkov, Y., Lavi, O., Goldberg, Y., 2016. Fine-grained analysis of sentence embeddings using auxiliary prediction tasks. *arXiv preprint arXiv:1608.04207*.
- Aggarwal, T., Singh, S., Awasthi, A., Kanade, A., Natarajan, N., 2025. Nextcoder: Robust adaptation of code lms to diverse code edits. In: *Forty-Second International Conference on Machine Learning*.
- Ahmad, W.U., Chakraborty, S., Ray, B., Chang, K.-W., 2021. Unified pre-training for program understanding and generation. *arXiv preprint arXiv:2103.06333*.
- Ahmed, T., Devanbu, P., 2022. Multilingual training for software engineering. In: *Proceedings of the 44th International Conference on Software Engineering*. pp. 1443–1455.
- Belinkov, Y., 2016. Probing classifiers: Promises, shortcomings, and advances. In: *Computational Linguistics*. pp. 1–12.
- Belinkov, Y., Durrani, N., Dalvi, F., Sajjad, H., Glass, J., 2017. What do neural machine translation models learn about morphology?. *arXiv preprint arXiv:1704.03471*.
- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners. <http://arxiv.org/abs/2005.14165> arXiv:2005.14165.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., d. O. Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chi, E.A., Hewitt, J., Manning, C.D., 2020. Finding universal grammatical relations in multilingual bert. *arXiv preprint arXiv:2005.04511*.
- Chomsky, N., 2014. *Aspects of the Theory of Syntax*. MIT Press, p. 11.
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*.
- Hewitt, J., Manning, C.D., 2019. A structural probe for finding syntax in word representations. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. pp. 4129–4138.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al., 2024. Qwen2. 5-Coder. technical report, *arXiv preprint arXiv:2409.12186*.
- Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., Brockschmidt, M., 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.
- Karmakar, A., Robbes, R., 2021. What do pre-trained code models know about code? In: *2021 36th IEEE/ACM International Conference on Automated Software Engineering*. ASE, IEEE, pp. 1332–1336.
- Karmakar, A., Robbes, R., 2023. Inspect: Intrinsic and systematic probing evaluation for code transformers. In: *IEEE Transactions on Software Engineering*.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al., 2023. StarCoder: may the source be with you!. *arXiv:2305.06161*.
- Limisiewicz, T., Mareček, D., 2020. Introducing orthogonal constraint in structural probes. *arXiv preprint arXiv:2012.15228*.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- López, J.A.H., Weyssow, M., Cuadrado, J.S., Sahraoui, H., 2022. Ast-probe: Recovering abstract syntax trees from hidden representations of pre-trained language models. *arXiv preprint arXiv:2206.11719*.
- Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al., 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. In: *Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.

- Ma, W., Liu, S., Zhao, M., Xie, X., Wang, W., Hu, Q., Zhang, J., Liu, Y., 2024. Unveiling code pre-trained models: Investigating syntax and semantics capacities. *ACM Trans. Softw. Eng. Methodol.* 33, 1–29.
- Manning, C., Schütze, H., 1999. *Foundations of Statistical Natural Language Processing*. MIT Press.
- Marcus, M., Santorini, B., Marcinkiewicz, M.A., 1993. Building a large annotated corpus of english: The penn treebank. In: *Computational Linguistics*. vol. 19, pp. 313–330.
- Min, B., Ross, H., Sulem, E., Veyseh, A.P.B., Nguyen, T.H., Sainz, O., Agirre, E., Heinz, I., Roth, D., 2021. Recent advances in natural language processing via large pre-trained language models: A survey. *arXiv preprint arXiv:2111.01243*.
- Mrini, K., Dérnoncourt, F., Tran, Q., Bui, T., Chang, W., Nakashole, N., 2019. Rethinking self-attention: Towards interpretability in neural parsing. *arXiv preprint arXiv:1911.03875*.
- Rabin, M.R.I., Hussain, A., Alipour, M.A., Hellendoorn, V.J., 2023. Memorization and generalization in neural code intelligence models. *Inf. Softw. Technol.* 153, 107066.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683*.
- Rogers, A., Kovaleva, O., Rumshisky, A., 2020. A primer in bertology: What we know about how bert works. *Trans. Assoc. Comput. Linguist.* 8, 842–866.
- Sanh, V., Debut, L., Chaumond, J., Wolf, T., 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
- Sennrich, R., Haddow, B., Birch, A., 2015. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*.
- Shen, D., Chen, X., Wang, C., Sen, K., Song, D., 2022. Benchmarking language models for code syntax understanding. *arXiv preprint arXiv:2210.14473*.
- Shen, Y., Lin, Z., Jacob, A.P., Sordani, A., Courville, A., Bengio, Y., 2018. Straight to the tree: Constituency parsing with neural syntactic distance. *arXiv preprint arXiv:1806.04168*.
- Tian, Y., Song, Y., Xia, F., Zhang, T., 2020. Improving constituency parsing with span attention. *arXiv preprint arXiv:2010.07543*.
- Troshin, S., Chirkova, N., 2022. Probing pretrained models of source code. *arXiv preprint arXiv:2202.08975*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems*. vol. 30.
- Wan, Y., Zhao, W., Zhang, H., Sui, Y., Xu, G., Jin, H., 2022. What do they capture?—a structural analysis of pre-trained language models for source code. *arXiv preprint arXiv:2202.06840*.
- Wang, Y., Wang, W., Joty, S., Hoi, S.C., 2021. Codet5: Identifier-aware unified pre-trained encoder–decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al., 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*.
- Xu, F.F., Alon, U., Neubig, G., Hellendoorn, V.J., 2022. A systematic evaluation of large language models of code. *arXiv preprint arXiv:2202.13169*.
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: *Advances in Neural Information Processing Systems*. pp. 10197–10207.
- Zhou, J., Zhao, H., 2019. Head-driven phrase structure grammar parsing on penn treebank. *arXiv preprint arXiv:1907.02684*.